

Enterprise Multi-Tier Web App Deployment

By Oyeleke Ismail

AWS Solutions Architect & .NET Developer

Using Amazon VPC, EC2, RDS, Secrets Manager, ALB, and Auto Scaling

1. Project Overview

This project demonstrates the deployment of an enterprise-grade **Multi-Tier Web Application** on AWS. Unlike simple single-server deployments, this architecture separates the application logic (Web Tier) from the data storage (Database Tier) to ensure scalability, security, and high availability.

The solution leverages a **Custom Virtual Private Cloud (VPC)** for network isolation, **Amazon EC2** for the web servers, **Amazon RDS** for managed database services, **AWS Secrets Manager** for secure credential management, and an **Application Load Balancer (ALB)** to distribute traffic efficiently.

2. Problem Statement

Monolithic applications hosted on a single server often face critical issues:

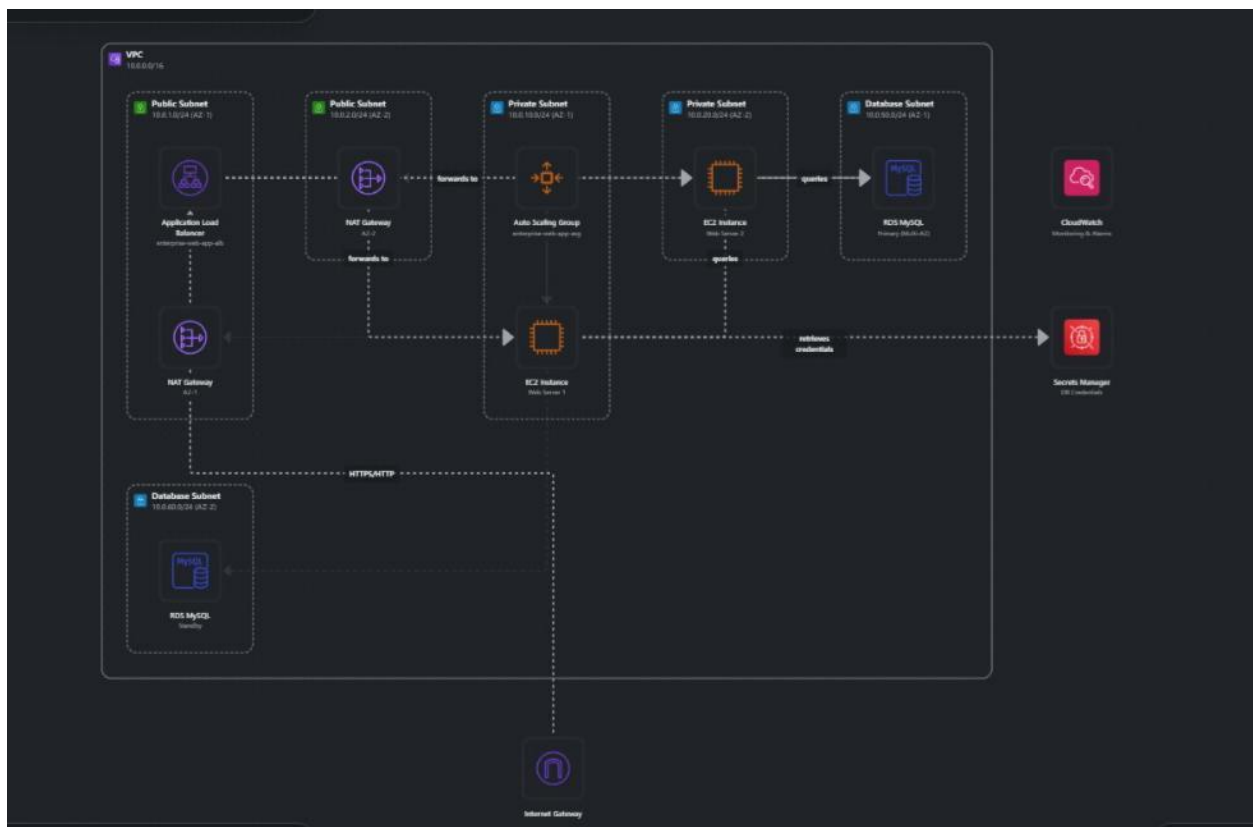
- **Single Point of Failure:** If the server crashes, the entire application goes offline.
- **Security Risks:** Hardcoding database passwords in scripts or storing them in plain text creates a major security vulnerability.
- **Scalability Limits:** Scaling a single server vertically (adding more CPU/RAM) is expensive and has limits.

This project addresses these challenges by implementing a **3-Tier Architecture** that decouples the web and database layers and automates security best practices.

3. Project Objectives

- **High Availability:** Ensure the application remains online even if a single server fails.
 - **Security:** Isolate the database in a private network and secure credentials using **Secrets Manager**.
 - **Scalability:** Implement Auto Scaling to handle varying traffic loads automatically.
 - **Automation:** Use User Data scripts to automate software provisioning.
-

4. Architecture Overview



- **Public Subnet:** Hosts the **Application Load Balancer (ALB)** and **NAT Gateway** for connectivity.
- **Private Subnet (Web Tier):** Hosts **EC2 Instances** running the WordPress application.
- **Private Subnet (Data Tier):** Hosts the **Amazon RDS (MySQL)** database.
- **Security Layer:** **AWS Secrets Manager** stores the database credentials, removing the need to manage passwords manually.

5. AWS Services Used

- **Amazon VPC:** Custom network environment (10.0.0.0/16).
- **Amazon EC2:** Virtual servers for the web application.
- **Amazon RDS:** Managed MySQL database service.
- **AWS Secrets Manager:** Securely encrypts, stores, and rotates database credentials.
- **Application Load Balancer (ALB):** Distributes incoming traffic.
- **Auto Scaling Group (ASG):** Automatically adds or removes servers.
- **NAT Gateway:** Allows private instances to download updates securely.
- **Security Groups:** Acts as a virtual firewall at instance level.

6. AWS Region

The solution was deployed in the **us-east-1 (N. Virginia)** AWS region.

7. Implementation Steps

Step 1: Network Setup (VPC)

I navigated to the VPC Dashboard and created a custom VPC named My-Enterprise-VPC with a CIDR block of 10.0.0.0/16.

- **Subnets:** Created Public Subnets for the Load Balancer and Private Subnets for the Application and Database tiers across two Availability Zones (us-east-1a, us-east-1b) for redundancy.
- **Gateways:** Attached an **Internet Gateway (IGW)** for public access and a **NAT Gateway** to allow private instances to download software updates.
- **Routing:** Configured Route Tables to direct public traffic to the IGW and private traffic to the NAT Gateway.

Your VPCs

VPCs | VPC encryption controls

Your VPCs (1/2) Info

Find VPCs by attribute or tag

Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	
My-Enterprise-VPC	vpc-052e3635fb95a3da8	Available	-	-	Off	1

Subnets (1/4) Info

Find subnets by attribute or tag

VPC: vpc-052e3635fb95a3da8

Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR
Private Subnet 1	subnet-09b70adb1c0be8a70	Available	vpc-052e3635fb95a3da8 My-...	Off	10.0.2.0/24
Public Subnet 2	subnet-0f5e902a0585b8e40	Available	vpc-052e3635fb95a3da8 My-...	Off	10.0.3.0/24
Public Subnet 1	subnet-001a847c377aa9cb1	Available	vpc-052e3635fb95a3da8 My-...	Off	10.0.1.0/24
Private Subnet 2	subnet-0f384d33651a11585	Available	vpc-052e3635fb95a3da8 My-...	Off	10.0.4.0/24

Step 2: Security Group Configuration (The Firewall)

Before launching any resources, I established the security architecture by creating three distinct Security Groups and "chaining" them together using Least Privilege principles.

1. **ALB SG:** Allows HTTP (80) from Anywhere (0.0.0.0/0).
2. **Web Tier SG:** Allows HTTP (80) **only** from the **ALB SG**.
3. **Database SG:** Allows MySQL (3306) **only** from the **Web Tier SG**.

Security Groups (4) Info

Find security groups by attribute or tag

VPC ID = vpc-052e3635fb95a3da8

Security group ID	Security group name	VPC ID	Description	Own
sg-0ef5654050db158f0	DB-SG	vpc-052e3635fb95a3da8	Allows inbound MySQL traffic on port 3...	7684
sg-097c6f2b399e92bc9	default	vpc-052e3635fb95a3da8	default VPC security group	7684
sg-0108745ac07e4c8a8	Web-Tier-SG	vpc-052e3635fb95a3da8	Allows inbound HTTP traffic specifically...	7684
sg-0f875073b04266172	ALB-SG	vpc-052e3635fb95a3da8	Allows inbound HTTP/HTTPS traffic fro...	7684

Step 3: Launch Web Tier (EC2 with Automation)

I launched an EC2 instance in the **Private Subnet**, attaching the **Web Tier SG**. Instead of manually installing software, I utilized a **User Data Script** to automate the provisioning of the LAMP stack (Linux, Apache, MySQL, PHP) and WordPress.

SCRIPT USED:

```
Bash

#!/bin/bash
dnf update -y
dnf install -y httpd wget php php-mysql php-json php-common php-devel
systemctl start httpd
systemctl enable httpd
cd /var/www/html
wget https://wordpress.org/latest.tar.gz
tar -xzf latest.tar.gz
cp -r wordpress/* .
rm -rf wordpress latest.tar.gz
chown -R apache:apache /var/www/html
chmod -R 755 /var/www/html
echo "Healthy" > /var/www/html/health.html
```

EC2 INSTANCE:

The screenshot displays the AWS Management Console interface. At the top, the navigation bar shows various services like S3, Route 53, EC2, VPC, Aurora and RDS, and CloudFront. The main content area is titled 'Instances (1/3) Info'. Below this, there's a search bar and a table listing three EC2 instances. The first instance, 'Web-Tier-Insta...', is selected. Below the table, the 'Details' tab for the selected instance 'i-0f3d2987c0e71f845' is shown. It includes fields for Instance ID, Public IPv4 address, Private IPv4 addresses, IPv6 address, Instance state (Running), and Public DNS.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
Web-Tier-Insta...	i-0f3d2987c0e71f845	Running	t2.micro	...	View alarms +	us-east-1a	-
	i-0317607186698a312	Running	t2.micro	...	View alarms +	us-east-1a	-
	i-0b4b0e0b6cc71bc91	Running	t2.micro	...	View alarms +	us-east-1b	-

i-0f3d2987c0e71f845 (Web-Tier-Instance)		
Details	Status and alarms	Monitoring
Instance summary		
Instance ID i-0f3d2987c0e71f845	Public IPv4 address -	Private IPv4 addresses 10.0.2.208
IPv6 address -	Instance state Running	Public DNS -

Step 4: Application Load Balancer (ALB)

I deployed an internet-facing **Application Load Balancer** in the Public Subnets and attached the **ALB SG**.

- **Target Group:** I registered the Web Tier EC2 instance as a target.
- **Health Check:** Configured the ALB to check /health.html to ensure traffic is only sent to healthy instances.

The screenshot displays the AWS Management Console interface for the 'EC2 > Load balancers' section. The console shows a list of load balancers with the following details:

Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones
My-Enterprise-ALB	Active	application	Internet-facing	IPv4	vpc-052e3635fb95a3da8	2 Availability Zones

Below the list, the 'Registered targets (3)' section is visible, showing three targets registered to the load balancer:

Instance ID	Name	Port	Zone	Health status	Health status details
i-0317607186698a312		80	us-east-1a (us...)	Healthy	-
i-0b4b0e0b6cc71bc91		80	us-east-1b (us...)	Healthy	-
i-0f3d2987c0e71f845	Web-Tier-Insta...	80	us-east-1a (us...)	Healthy	-

Step 5: Deploy Database (RDS) with Secrets Manager

I provisioned a highly available **Amazon RDS (MySQL)** database in the **Private Subnet**.

- **Connectivity:** I selected the **Database SG** to strictly limit access to the Web Tier.
- **Secrets Management:** Instead of manually creating a password, I enabled "**Manage master credentials in AWS Secrets Manager**". This automatically generated a high-entropy password and stored it securely in the Secrets Manager vault.

Settings

DB instance identifier [Info](#)
Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.

wordpress-db

The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

▼ **Credentials Settings**

Master username [Info](#)
Type a login ID for the master user of your DB instance.

admin

1 to 16 alphanumeric characters. The first character must be a letter.

Credentials management
You can use AWS Secrets Manager or manage your master user credentials.

☒ **Managed in AWS Secrets Manager - most secure**
RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.

☐ **Self managed**
Create your own password or have RDS create a password that you manage.

Info If you manage the master user credentials in AWS Secrets Manager, additional charges apply. See [AWS Secrets Manager pricing](#). Additionally, some RDS features aren't supported. See [limitations](#).

Step 6: Credential Retrieval

I navigated to the **AWS Secrets Manager** console to retrieve the database credentials securely.

- I verified that the secret was successfully created and associated with the RDS database.
- I retrieved the username and password values from the secret to use them during the application configuration, ensuring I never had to write them down or store them in plain text files.

Overview | Rotation | Versions | Replication | Tags

Secret value [Info](#) [Close](#) [Edit](#)

Retrieve and view the secret value.

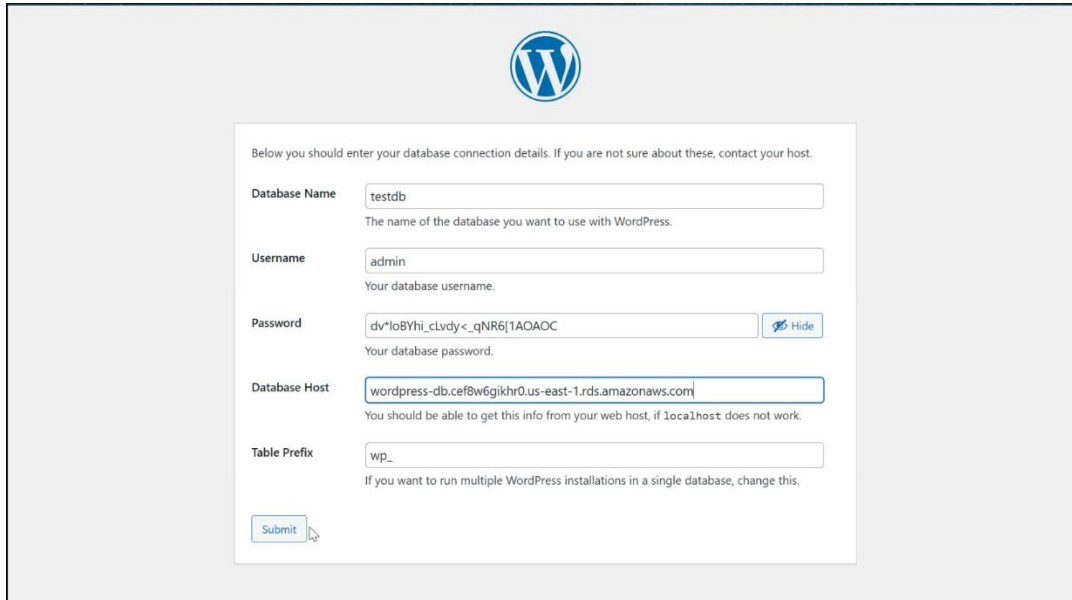
Key/value | Plaintext

Secret key	Secret value
username	admin
password	dv*1oBYhi_cLvdy<qNR6[1AOAOC

Step 7: Application Deployment

I accessed the application via the **ALB DNS Name**.

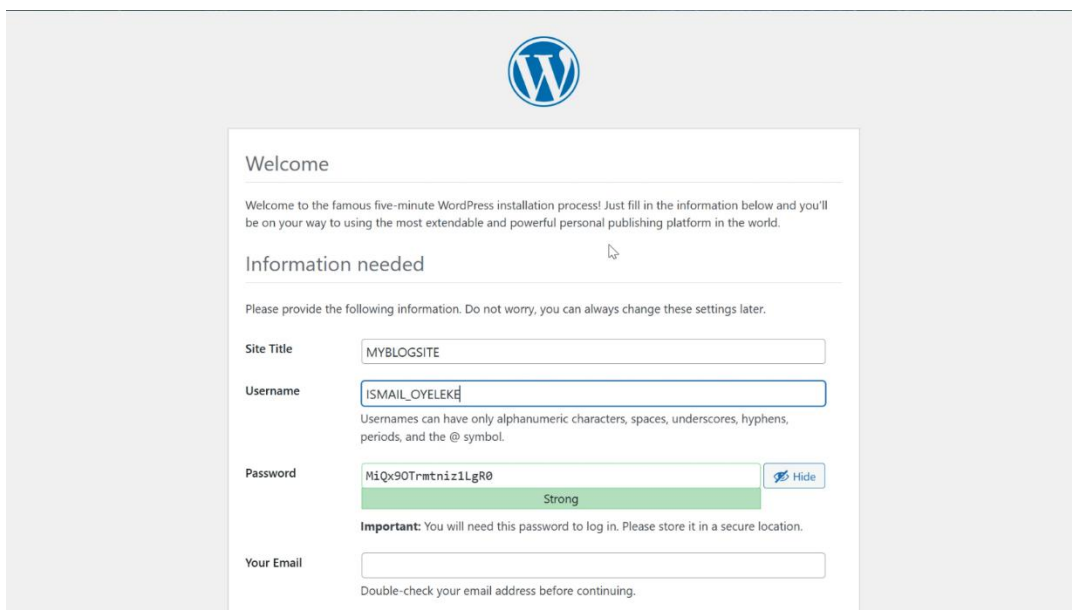
- I connected the application to the database using the **RDS Endpoint** and the credentials securely retrieved from **Secrets Manager**.
- I completed the WordPress installation, finalizing the connection between the Web and Data tiers.



The image shows the WordPress database connection screen. At the top is the WordPress logo. Below it, a message states: "Below you should enter your database connection details. If you are not sure about these, contact your host." The form contains the following fields:

- Database Name:** testdb. Subtext: "The name of the database you want to use with WordPress."
- Username:** admin. Subtext: "Your database username."
- Password:** dv*loBYhi_clvdy<_qNR6[1AOAOC. Subtext: "Your database password." There is a "Hide" button with an eye icon.
- Database Host:** wordpress-db.ccf8w6gikhr0.us-east-1.rds.amazonaws.com. Subtext: "You should be able to get this info from your web host, if localhost does not work."
- Table Prefix:** wp_. Subtext: "If you want to run multiple WordPress installations in a single database, change this."

A "Submit" button is at the bottom left.



The image shows the WordPress "Welcome" screen. At the top is the WordPress logo. Below it, a message states: "Welcome to the famous five-minute WordPress installation process! Just fill in the information below and you'll be on your way to using the most extendable and powerful personal publishing platform in the world." The section is titled "Information needed" and includes the instruction: "Please provide the following information. Do not worry, you can always change these settings later."

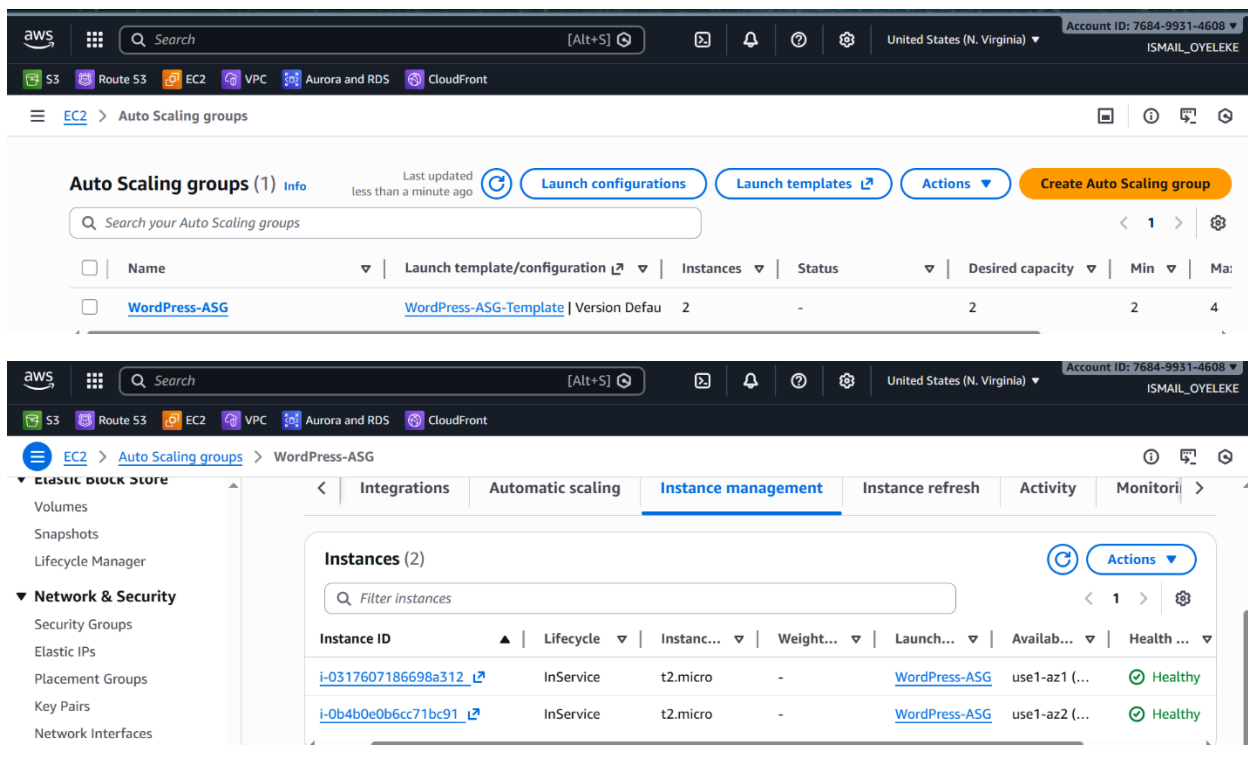
The form contains the following fields:

- Site Title:** MYBLOGSITE
- Username:** ISMAIL_OYELEKE. Subtext: "Usernames can have only alphanumeric characters, spaces, underscores, hyphens, periods, and the @ symbol."
- Password:** MiQx90Trmt niz1LgR0. Subtext: "Important: You will need this password to log in. Please store it in a secure location." There is a "Hide" button with an eye icon. A green bar below the password indicates the strength is "Strong".
- Your Email:** (empty field). Subtext: "Double-check your email address before continuing."

Step 8: Auto Scaling & High Availability

To ensure resilience, I created an **Auto Scaling Group (ASG)**.

- **Launch Template:** Created an AMI from the configured Web Instance and selected the **Web Tier SG**.
- **Policy:** Configured the ASG to maintain a desired capacity of **2 instances**, scaling up if CPU utilization exceeds 50%.
- **Testing:** I manually terminated an instance to verify that the ASG automatically detected the failure and launched a replacement.



8. Security Considerations

- **Secrets Management:** Database credentials are **never hardcoded**. They are managed by AWS Secrets Manager, which can automatically rotate passwords, significantly reducing the risk of credential theft.
- **Private Isolation:** The application and database servers reside in private subnets with **no public IP addresses**.
- **Least Privilege:** Security Groups are tightly scoped to allow traffic only from trusted sources.

9. Challenges and Solutions

Challenge: Managing Sensitive Database Credentials

- *Issue:* Storing database passwords in plain text or scripts is a major security violation.
- *Solution:* I integrated **AWS Secrets Manager** with RDS. This offloaded the responsibility of password complexity and storage to a managed service, ensuring compliance with security best practices.

Challenge: Load Balancer Availability Zones

- *Issue:* The ALB requires subnets in at least two Availability Zones (AZs) to function.
 - *Solution:* I created a secondary Public Subnet in us-east-1b to satisfy the ALB requirement.
-

10. Project Outcome

I successfully architected and deployed a **fault-tolerant, scalable 3-Tier Web Application**. The system automatically handles traffic spikes via Auto Scaling and self-heals in the event of server failure. By using **AWS Secrets Manager**, I demonstrated an enterprise-level approach to security, ensuring that sensitive credentials are encrypted and managed automatically.

11. GitHub Repository

Here is the GitHub Repository for this project:

<https://github.com/ISMAIL-OYELEKE/Project-3-Enterprise-Multi-Tier-Web-App-Deployment>

12. Conclusion

This project moves beyond basic hosting to demonstrate **Enterprise Cloud Architecture**. By combining manual VPC design, security group chaining, Secrets Manager integration, and automated scaling, I have built a robust infrastructure capable of supporting critical business applications with minimal downtime.